# Laporan Progress Tugas 2 (UTS) Sistem Terdistribusi

Implementasi dan Analisis Komunikasi Message Passing, Remote Method Invocation, dan Publish-Subscribe

Disusun oleh: [nama Orang A] dan [nama Orang B]

## 1. Keterangan Umum

Tugas dikerjakan oleh dua orang. Pada tahap progress ini, setiap anggota mengerjakan bagiannya masing-masing di **satu perangkat (localhost)**. Pengujian antar dua perangkat belum dilakukan dan akan dilakukan setelah implementasi masing-masing selesai.

## 2. Pembagian Pengerjaan

| Anggota | Bagian |
|---|---|
| Orang A [nama] | Message passing (socket TCP), kemudian RMI (Pyro5) |
| Orang B [nama] | Publish-subscribe (MQTT), kemudian persiapan alat pengujian |

Teknologi yang disepakati: Python 3, broker Mosquitto, library paho-mqtt, dan Wireshark untuk analisis paket.

## 3. Progress Orang B (Publish-Subscribe)

### 3.1 Implementasi

- Broker Mosquitto terpasang dan berjalan di port 1883.
- `publisher.py`: mengirim data berukuran 200-1000 KB dengan pilihan QoS MQTT 0 atau 1, jumlah pengulangan, dan jeda antar kiriman.
- `subscriber.py`: menerima pesan dan mencatat ukuran serta waktu terima, dengan opsi `--delay` untuk mensimulasikan subscriber yang lambat.
- `generator.py`: membuat data uji acak berukuran 200, 400, 600, 800, dan 1000 KB.

### 3.2 Hasil Pengujian di Localhost

#### 3.2.1 Pengujian pengiriman data (200-1000 KB)

Skenario: publisher mengirim satu pesan teks acak per ukuran ke topik `uts/data` dengan MQTT QoS 1. Broker dan subscriber berada di localhost.

| Ukuran data | Diterima subscriber | Waktu publish sampai PUBACK broker |
|---|---|---|
| 200 KB | Ya | 0,001 s |
| 400 KB | Ya | 0,002 s |
| 600 KB | Ya | 0,003 s |
| 800 KB | Ya | 0,004 s |
| 1000 KB | Ya | 0,005 s |

**Analisis:** kelima ukuran data diterima utuh oleh subscriber dan tidak ada pesan yang gagal. Waktu kirim naik seiring ukuran data. Nilainya sangat kecil karena pengujian dilakukan di localhost tanpa

jaringan fisik, sehingga **belum merepresentasikan delay jaringan**.

[Sisipkan screenshot: terminal publisher dan subscriber berdampingan]

### 3.2.2 Pengujian marshaling

Skenario: capture Wireshark pada interface loopback dengan filter `tcp.port == 1883`, mengirim teks "Halo dari publisher" dengan QoS 0 dan QoS 1.

| Pertanyaan | Hasil pengamatan |
|---|---|
| Protokol layer transport | TCP (port tujuan 1883) |
| Protokol layer application | MQTT |
| ACK di layer application | Ada pada QoS 1: paket *Publish Ack* (Header Flags 0x40, Msg Len 2) dengan Message Identifier yang sama dengan Publish. Pada QoS 0 tidak ada ACK. |
| Teks dapat dibaca | Ya, plaintext (tanpa TLS). Teks payload dan nama topik *uts/data* terbaca di hex dump. |
| Format data | Biner dengan struktur tetap: fixed header (0x32 = PUBLISH, QoS 1), remaining length, panjang dan nama topik, message identifier (hanya QoS 1), lalu payload berupa teks mentah. |

Setiap pesan tampak dua kali di capture: sekali dari publisher ke broker, lalu dari broker diteruskan ke subscriber. Hal ini menunjukkan broker berperan sebagai perantara pub-sub. Alamat yang tercatat adalah ::1 (IPv6 loopback).

[Sisipkan screenshot Wireshark: frame Publish Message QoS 1 (panel detail TCP + MQTT dan hex dump), frame Publish Ack, dan frame Publish QoS 0 sebagai pembanding]

### 3.2.3 Pengujian synchronous / asynchronous

Skenario: publisher mengirim data 200 KB sebanyak **5 kali** berturut-turut (QoS 1). Subscriber diberi sleep() sebelum selesai memproses setiap pesan: 1, 5, dan 10 detik.

| Delay subscriber | Publisher selesai mengirim 5 pesan | Subscriber selesai memproses 5 pesan |
|---|---|---|
| 1 detik | 0,003 s | 5 detik (10:48:48 - 10:48:53) |
| 5 detik | 0,003 s | 25 detik (10:48:57 - 10:49:22) |
| 10 detik | 0,004 s | 50 detik (10:49:30 - 10:50:20) |

**Analisis:** waktu publisher mengirim kelima pesan hampir tidak berubah (sekitar 0,003-0,004 detik) walaupun delay subscriber dinaikkan dari 1 ke 10 detik. Jadi lambatnya subscriber tidak mempengaruhi jeda request berikutnya di publisher. Subscriber memproses pesan satu per satu, sehingga waktu totalnya sama dengan 5 x delay.
Hal ini terjadi karena publisher hanya menunggu konfirmasi (PUBACK) dari **broker**, bukan dari subscriber. Broker menampung pesan lalu meneruskannya ke subscriber. Dengan demikian pub-sub bersifat **asynchronous**: publisher dan subscriber tidak terikat secara waktu.

[Sisipkan 3 screenshot: terminal publisher dan subscriber untuk delay 1, 5, dan 10 detik]

## 3.3 Yang Belum Dikerjakan

- Pengujian QoS resmi (delay, throughput, jitter, packet loss). Nilai di localhost tidak merepresentasikan jaringan nyata, sehingga pengujian dilakukan setelah dua perangkat siap.
- Pengujian pub-sub antar dua perangkat (broker dan subscriber di mesin berbeda).

## 4. Progress Orang A

[Diisi oleh Orang A: status implementasi message passing dan RMI, hasil uji di localhost, screenshot]

## 5. Rencana Selanjutnya

- Menyelesaikan implementasi yang tersisa di masing-masing perangkat.
- Menggabungkan kode dan mengubah alamat IP agar berjalan di dua perangkat.
- Melakukan tiga pengujian resmi: QoS, marshaling, dan sync/async.
- Menyusun laporan akhir sesuai template.

## Lampiran: Template Tabel QoS (diisi setelah pengujian 2 perangkat)

**Tabel 1. QoS Message Passing**

| Ukuran data (KB) | Delay (ms) | Throughput (KB/s) | Jitter (ms) | Packet loss (%) |
|---|---|---|---|---|
| 200 | | | | |
| 400 | | | | |
| 600 | | | | |
| 800 | | | | |
| 1000 | | | | |

**Tabel 2. QoS RMI**

| Ukuran data (KB) | Delay (ms) | Throughput (KB/s) | Jitter (ms) | Packet loss (%) |
|---|---|---|---|---|
| 200 | | | | |
| 400 | | | | |
| 600 | | | | |
| 800 | | | | |
| 1000 | | | | |

**Tabel 3. QoS Publish-Subscribe (MQTT)**

| Ukuran data (KB) | Delay (ms) | Throughput (KB/s) | Jitter (ms) | Packet loss (%) |
|---|---|---|---|---|
| 200 | | | | |
| 400 | | | | |
| 600 | | | | |
| 800 | | | | |
| 1000 | | | | |

**Cara mengisi dari Wireshark**

| Parameter | Sumber |
|---|---|
| Delay | Rata-rata selisih waktu antar paket, atau waktu dari PUBLISH sampai PUBACK |
| Throughput | Total byte dibagi durasi (Statistics > Capture File Properties) |
| Jitter | Variasi (selisih) delay antar paket yang berurutan |

| Parameter | Sumber |
| --- | --- |
| Packet loss | Persentase paket hilang atau retransmisi (filter `tcp.analysis.retransmission`) |